

AI-Driven Personalized Learning and Remedial Recommendation

The project explained simply, plus questions to prepare for the final review

Based on: N. Pradeesh, M. G. Thushara, K. Arun Krishna, V. Pranav and S. Krishnamoorthy, "AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation", *IEEE Access*, vol. 13, 2025.

What is inside

1. The project in 30 seconds
2. The problem it solves
3. How it works, step by step
4. Each part explained simply
5. Understanding the demo output
6. Results and what they mean
7. Technologies and code files
8. How to run it
9. Final review: questions and answers
10. Tips for the review day

1. The project in 30 seconds

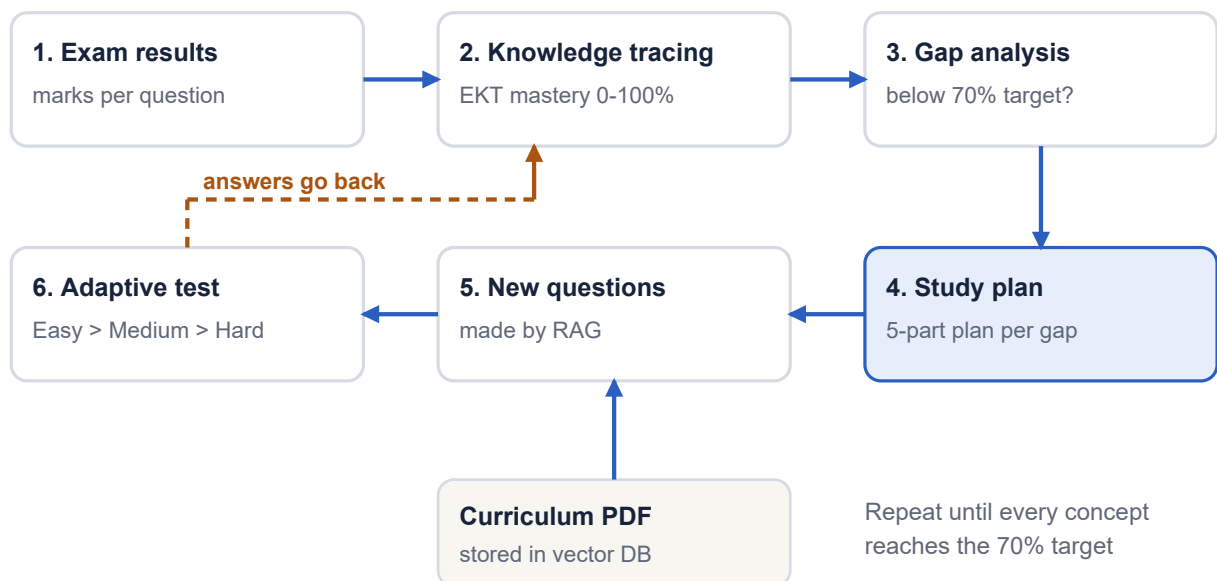
A teacher gives an exam. Our system looks at each student's marks and finds the **topics (concepts)** the student is weak in. For every weak topic it writes a **personal study plan**, then makes **new practice questions** from the syllabus and gives an **adaptive test** (Easy, then Medium, then Hard). The test answers go back into the system, it checks again, and the loop repeats until the student reaches the target level.

Everything is decided using **Outcome-Based Education (OBE)** rules: a **threshold of 50%** (minimum pass) and a **target of 70%** (good mastery). Anything below 70% is called a **knowledge gap**.

2. The problem it solves

Problem with normal exams	How this project helps
Writing questions by hand is slow and can be biased.	AI writes questions from the syllabus, and other AI models check them.
Everyone gets the same test, even though students learn at different speeds.	Each student gets questions only on their own weak topics, at their own level.
In a big class, personal feedback comes late or never.	A study plan is generated immediately for every weak topic, for every student.
A total mark does not say <i>which</i> topic is weak.	Marks are analysed topic by topic (concept-level evaluation).

3. How it works, step by step



Step	What happens (simple words)
Step 0 - Read the syllabus (done once)	The curriculum PDF is cut into small pieces (chunks), each piece is turned into numbers (an embedding), and all pieces are stored in a vector database. The AI also lists the concepts (one per unit). The teacher checks this list.
Step 1 - Exam results come in	For each student: which question, which concept, maximum marks, marks obtained.
Step 2 - Knowledge tracing (EKT)	A neural network reads the student's answers in order and estimates mastery for every concept, from 0% to 100%.
Step 3 - Gap analysis	Mastery below 70% means a gap. Level: below 50% Beginner, 50-70% Intermediate, 70% and above Expert.
Step 4 - Study plan	For each gap, the main AI model (OpenAI) writes a plan with 5 parts. Two other AI models (Gemini and DeepSeek) give it a score. If the score is low, it is rewritten.
Step 5 - New questions	The AI takes the concept's syllabus text from the vector database and writes multiple-choice questions. Duplicates are removed and the other models check the difficulty.
Step 6 - Adaptive test	The student starts at Easy. Only after passing Easy do they get Medium, then Hard.
Step 7 - Loop	The test answers are added to the student's history and Step 2 runs again. The loop stops when every concept reaches 70% (or after the maximum number of rounds).

4. Each part explained simply

OBE: threshold and target

OBE means the course is designed around outcomes the student must achieve. Every outcome has a **threshold** (50%, the minimum) and a **target** (70%, the expected level). We use these two numbers to decide gaps and levels.

Where in the code: `config.yaml`, `gap.py`

RAG (Retrieval-Augmented Generation)

Before the AI writes anything, we **retrieve** the matching part of the syllabus and give it to the AI. So the AI writes **from our syllabus**, not from its own memory. This stops out-of-syllabus or made-up questions.

Where in the code: `rag.py`

Chunks, embeddings, vector database

The syllabus is cut into pieces of about 1000 characters, with 100 characters overlapping so no sentence is lost at the edge. Each piece becomes a list of 384 numbers (an **embedding**) using the all-MiniLM-L6-v2 model. Pieces with similar meaning get similar numbers. FAISS (the vector database) quickly finds the pieces closest to a topic.

Where in the code: `rag.py`, `embeddings.py`

Knowledge tracing: EKT

Knowledge tracing means "tracking what a student knows over time". EKT (Exercise-aware Knowledge Tracing) reads each answer **and the text of the question**, and passes them through an **LSTM**, a neural network with memory. At the end it outputs one mastery value (0-1) for every concept.

Where in the code: `kt.py`

DKT (the comparison model)

Deep Knowledge Tracing is the older model. It only sees "concept number + right/wrong" and never reads the question text. We train both and compare them.

Where in the code: `kt.py`

Remedial study plan (5 parts)

1) Learning objectives, 2) Topics to revise, 3) Remedial explanation, 4) Practice activities, 5) Concept gap rationale, which tells the student *why* this plan was given (their mastery and the questions they missed).

Where in the code: `remedial.py`

Cross-verification

One AI model writes; two **different** AI models check it and give a score from 1 to 5. If the average is below 3, the plan or question is rewritten or rejected. Different models catch different mistakes.

Where in the code: `remedial.py`, `qgen.py`

MCQ generation and duplicates

For each level we ask for about 3 times the questions we need. The same model then fixes grammar. We compare every new question with all saved questions. If the **cosine similarity is above 0.85** (very similar meaning), it is a duplicate and is dropped.

Where in the code: `qgen.py`

Bloom's Taxonomy

Levels of thinking. Easy questions: Remember and Understand. Medium: Apply and Analyze. Hard: Evaluate and Create.

Where in the code: `llm.py`, `qgen.py`

Adaptive test and the rule "5 and 80"

"5 and 80" means 5 questions per level, and you need 80% (4 of 5) to move up. The category is the highest level passed: none = Not Competent, Easy = Beginner, Medium = Intermediate, Hard = Expert.

Where in the code: `evaluation.py`

Offline mock AI

If no API key is given, a built-in "mock" model answers in the same format, so the project runs without internet or cost. With real keys in the `.env` file, the real models are used.

Where in the code: `llm.py`

5. Understanding the demo output

When you run `python -m plrs demo`, it prints 5 stages. This is what each one means:

Printed stage	What it means
1/5 Generating synthetic OBE data	Creates practice data: <code>historical.csv</code> (600 students, used to train EKT) and <code>current_batch.csv</code> (262 students, 16 questions, 5 concepts).
2/5 Ingesting curriculum	Stores the syllabus in the vector database (11 chunks) and finds 5 concepts.
3/5 Training EKT	Trains the model for 12 epochs. <code>test_auc 0.827</code> means the model predicts right/wrong answers well.
4/5 Knowledge-gap analysis	Shows, for student <code>CSE230001</code> , the exam score, the EKT mastery, the level, and YES if it is a gap.
5/5 Personalized learning cycle	Round 1: all 5 concepts are gaps, so 5 study plans are printed and a test is taken. Round 2: all mastery values are 0.74 or more, so it prints "Gaps: none - target reached".

A question the panel may ask: "Operators scored 75% in the exam, but mastery is only 43%. Why is it still a gap?"

Answer: The student answered only **one** question on Operators. One answer is not enough proof, so EKT keeps the estimate close to the class average. After the student answers more questions in the adaptive test, mastery rose to 75% in round 2. This is correct knowledge-tracing behaviour, and the paper also decides gaps using mastery, not the raw exam score.

6. Results and what they mean

Knowledge tracing: EKT vs DKT

Model	Test AUC (higher is better)	Test loss (lower is better)	Test MAE (lower is better)
EKT	0.827	0.504	0.331
DKT	0.665	0.637	0.447

AUC in simple words: if you pick one correct answer and one wrong answer at random, AUC is the chance that the model gives the correct one a higher score. 0.5 means guessing, 1.0 means perfect. EKT is better because it also reads the question text. The paper found the same ranking (EKT 0.933 on its real data). Ours is lower because our data is small and synthetic.

The learning loop (demo, student `CSE230001`)

Concept	Round 1 mastery	Round 2 mastery
Algorithmic problem solving and arrays	55%	85%
Data structures, memory and files	43%	90%
Fundamentals and system architecture	31%	79%
Operators, pointers and control	43%	75%

Concept	Round 1 mastery	Round 2 mastery
Strings and text processing	35%	74%

Simulated study with 30 students (paired t-test)

Concept	Before	After	p-value	Significant?
Fundamentals and system architecture	71.1	86.2	0.0014	yes
Operators, pointers and control	72.8	78.6	0.23	no
Algorithmic problem solving and arrays	79.4	87.5	0.06	no
Strings and text processing	66.7	92.5	below 0.0001	yes
Data structures, memory and files	67.8	88.1	below 0.0001	yes

Paired t-test in simple words: we compare the same students before and after. The **p-value** is the chance that such an improvement happened just by luck. Below 0.05 means the improvement is real (significant). Scores rose on all five concepts, and three were significant.

Be honest in the review: these students and their learning gain are **simulated**. The numbers show that the pipeline works correctly, not a real classroom result. The paper's real classroom study found p below 0.0001 on all five concepts.

7. Technologies and code files

Technology	Used for
Python 3.10+	The whole project
PyTorch	The LSTM neural networks for EKT and DKT
NumPy	Vector maths and metrics
pypdf	Reading the curriculum PDF
sentence-transformers (all-MiniLM-L6-v2)	Turning text into embeddings
FAISS	Vector database for fast similarity search
OpenAI GPT-4o-mini	Main generator: concepts, study plans, questions
Google Gemini 1.5 Flash, DeepSeek	Cross-verification (checking)
FastAPI + Uvicorn	REST API and web server
HTML + JavaScript	The web frontend
pytest	28 automated tests
YAML	config.yaml holds all settings

File	What it does
plrs/rag.py	Reads the syllabus, makes chunks, vector database, finds concepts
plrs/kt.py	EKT and DKT models: training and mastery
plrs/gap.py	Threshold/target check, levels, gaps
plrs/remedial.py	5-part study plans with cross-verification
plrs/qgen.py	MCQ generation, duplicate removal, question bank
plrs/evaluation.py	Adaptive test (Easy, Medium, Hard) and categories
plrs/pipeline.py	Runs the full loop (Algorithm 1 of the paper)
plrs/llm.py	Connects to OpenAI, Gemini, DeepSeek, or the offline mock
plrs/metrics.py	Similarity, readability, diversity, t-test
plrs/api.py, plrs/static/index.html	Web server and web page
plrs/cli.py	All the python -m plrs commands

8. How to run it

Command	What it does
<code>pip install -r requirements.txt</code>	Install everything (once)
<code>python -m plrs demo</code>	Run the whole project end to end
<code>python -m plrs serve</code>	Start the website: open http://127.0.0.1:8000/

Command	What it does
<code>python -m plrs experiments</code>	Produce the result tables
<code>python -m pytest -q</code>	Run the 28 tests

9. Final review: questions and answers

Short answers you can say in your own words. Practise saying them out loud.

A. Basics

Q1. What is your project in one line?

It finds which topics each student is weak in, gives a personal study plan, and tests them again with new questions, repeating until they reach the target.

Q2. Which paper is it based on?

"AI-Driven Personalized Learning and Remedial Recommendation Through Knowledge Concept-Centric Evaluation" by Pradeesh et al., IEEE Access, 2025.

Q3. What is OBE?

Outcome-Based Education. The course is built around outcomes students must achieve. Each outcome has a threshold (50%) and a target (70%).

Q4. What is a "concept" here?

One topic of the course, usually one unit of the syllabus, for example "String Manipulation and Text Processing". Every exam question is linked to a concept.

Q5. Who uses the system?

Students (see their gaps, study plans and tests) and teachers (upload the syllabus, check the concept list, set the test rule such as "5 and 80").

Q6. What are the four modules?

Knowledge gap identification, remedial recommendation, adaptive question generation and personalized evaluation.

B. Knowledge tracing and machine learning

Q7. What is knowledge tracing?

Estimating how well a student knows each concept from the order of their right and wrong answers, and updating it after every answer.

Q8. What is EKT and how is it different from DKT?

EKT is Exercise-aware Knowledge Tracing. It reads the text of each question as well as right/wrong. DKT only sees the concept number and right/wrong. So EKT understands the questions better.

Q9. Explain the EKT model structure.

Question text becomes an embedding and is combined with a concept embedding (size 100). It goes into a "right" or "wrong" half depending on the answer. An LSTM with 100 units keeps the knowledge state. A sigmoid layer outputs mastery for every concept. Settings: dropout 0.3, Adam optimizer, learning rate 0.001, batch 32, 12 epochs, 75/25 split.

Q10. Why an LSTM?

Learning happens in order. An LSTM is a neural network with memory, so it remembers earlier answers when it reads the next one.

Q11. What is AUC?

The chance that the model scores a correct answer higher than a wrong one. 0.5 is guessing, 1.0 is perfect. Our EKT got 0.827, DKT got 0.665.

Q12. Why is your AUC lower than the paper's 0.933?

The paper used real data from 1,276 students. We used a smaller synthetic dataset. The important point is that the ranking is the same: EKT beats DKT.

Q13. How is a gap decided?

If EKT mastery for a concept is below the 70% target, it is a gap. Below 50% is Beginner, 50-70% Intermediate, 70% and above Expert.

Q14. Why can mastery be low when the exam score is high?

When a student answered only one question on a concept, one answer is weak evidence, so the model stays near the class average. More answers make the estimate stronger.

C. RAG, questions and study plans

Q15. What is RAG and why do you use it?

Retrieval-Augmented Generation. We first find the matching syllabus text, then give it to the AI to write from. This keeps questions inside the syllabus and reduces made-up content.

Q16. What is an embedding? Why all-MiniLM-L6-v2?

An embedding is a list of numbers that represents the meaning of a text; similar meanings get similar numbers. MiniLM is small, fast and accurate, and it is the model the paper used.

Q17. What is a vector database? Why FAISS?

A database that finds the stored texts whose embeddings are closest to a query. FAISS does this very fast.

Q18. Why chunk size 1000 with overlap 100?

These are the values from the paper. Small chunks give precise search results, and the overlap makes sure a sentence cut at the edge still appears whole in one chunk.

Q19. How do you avoid duplicate questions?

Every new question is compared with all saved questions using cosine similarity. Above 0.85 means almost the same meaning, so it is dropped.

Q20. What is cross-verification?

OpenAI writes, and Gemini and DeepSeek each check and score it from 1 to 5. A low score means rewrite or reject. Two different models catch mistakes that one model would miss.

Q21. Why is OpenAI the main model?

In the paper's comparison (Table 3), OpenAI's questions were the most similar to teacher-written questions, so it was chosen as the main generator and the other two became checkers.

Q22. What is Bloom's Taxonomy and how do you use it?

Levels of thinking. Easy questions test Remember and Understand, Medium test Apply and Analyze, Hard test Evaluate and Create.

Q23. What are the five parts of a study plan?

Learning objectives, topics to revise, remedial explanation, practice activities, and concept gap rationale, which explains why the plan was given.

D. Adaptive test and the loop

Q24. How does the adaptive test work?

The student starts at Easy. They move to Medium only after passing Easy, then to Hard. The category is the highest level passed.

Q25. What does "5 and 80" mean?

5 questions per level, and 80% (4 of 5) needed to pass. For optional courses a teacher can use "1 and 100".

Q26. When does the loop stop?

When every concept reaches 70% mastery, or after the maximum number of rounds (3 by default).

Q27. Does the student see the same questions again?

No. Questions already shown to the student are excluded, so the next test uses new questions. This checks real understanding, not memory.

E. Results, data and testing

Q28. Is your data real?

No. It is synthetic, made by a simulator based on Item Response Theory (IRT): each simulated student has an ability per concept, and harder questions are less likely to be answered correctly. The code accepts real college data in the same CSV format.

Q29. What is a paired t-test and p-value?

It compares the same students before and after. The p-value is the chance the improvement happened by luck; below 0.05 means it is significant.

Q30. Why were two concepts not significant?

Only 30 simulated students, and those two concepts had high scores already before remediation, so there was less room to improve.

Q31. How did you test the code?

28 automated pytest tests cover every module: chunking, EKT training, gap rules, question de-duplication, study plans, adaptive test, the full loop and the web API. All pass.

F. Practical and future

Q32. What happens without API keys?

A built-in offline mock model answers in the same format, so everything runs without internet or cost. With keys in the .env file, real models are used.

Q33. How would a college use it with real data?

Upload the syllabus PDF, check the concept list, and upload the exam results CSV. The concept names in the CSV must match the concept list.

Q34. What is the frontend built with?

Plain HTML and JavaScript served by FastAPI. It calls the REST API: /analyze, /recommend and /evaluation.

Q35. What did you add beyond the paper?

Splitting the syllabus by unit before chunking (keeps questions on topic), an extra training loss so EKT's mastery values are reliable, an offline mode, a web frontend and automated tests.

Q36. What are the limitations?

Synthetic data; only EKT and DKT (the paper also tested DKVMN, AKT and SimpleKT); the offline mock writes weak questions, so real API keys are needed for good question quality.

Q37. What is the future work?

Test with real student data and real before/after tests, add the other knowledge-tracing models, and add a teacher screen to review flagged questions.

Q38. How does it support SDG 4?

SDG 4 is quality education for all. The system gives every student personal help at scale, including students who are struggling.

Q39. What about fairness and privacy?

Questions come only from the syllabus, several models cross-check, teachers review, every plan explains why it was given, and student records can be anonymised.

10. Tips for the review day

A 1-minute introduction you can say

"Our project is based on an IEEE Access 2025 paper. Normal exams give one mark, but they don't tell a student which topic is weak. Our system takes exam results, uses a neural network called EKT to estimate mastery for each topic, and flags every topic below the 70% OBE target. For each weak topic, an AI model writes a study plan from our syllabus using RAG, and two other AI models check it. Then the system makes new questions and gives an adaptive test from Easy to Hard. The answers go back into EKT, and the loop repeats until the student reaches the target. In our demo, all five concepts reached the target in two rounds, and EKT predicted answers better than the older DKT model."

A live demo in 5 steps

- Run `python -m plrs serve` before the review starts (models take about a minute to load).
- Open `http://127.0.0.1:8000/` and load sample learner CSE230001.
- Click **Analyse & get recommendations**: show the mastery bars and the gaps.
- Open one study plan and point at the 5 parts, especially "why this was recommended".
- Take the self-assessment, then click **Next round** and show mastery going up.

Things to say honestly

- The data is synthetic; the code accepts real data in the same format.
- Without API keys the offline mock is used; real keys give better questions.
- The t-test uses simulated students, so it shows the pipeline works, not a classroom result.

Numbers to remember

Number	Meaning
50% / 70%	OBE threshold / target
0.827 vs 0.665	Test AUC of EKT vs DKT
1000 / 100	Chunk size / overlap (characters)
0.85	Similarity above which a question is a duplicate
3x	Questions generated compared with the number needed
5 and 80	Default test rule: 5 questions per level, 80% to pass
100 / 0.3 / 12	EKT embedding and LSTM size / dropout / epochs
28	Automated tests, all passing